Documentatie algoritmi de sortare

David Tudor

April 2026

Cuprins

1 Introducere **2**

2 Istoricul Algoritmilor de Sortare **2**
2.1 Bubble Sort 2
2.2 Selection Sort 2
2.3 Insertion Sort 2
2.4 Merge Sort 3
2.5 Quick Sort 4

3 Structura Proiectului **4**

4 Rezultate Benchmark **5**

5 Analiza Anomaliilor **5**
5.1 Anomalia Quick Sort pe date „Flat” 5
5.2 Anomalia Insertion Sort pe date „Flat” 5

6 Concluzii **5**

7 Referințe **6**

1 Introducere

Acest proiect reprezintă o aplicație de performanță dedicată analizei computaționale a cinci algoritmi clasici de sortare. Obiectivul principal este evidențierea diferențelor pragmatice de execuție între complexitățile asimptotice $O(n^2)$ și $O(n \log n)$, utilizând diverse distribuții de date pentru a observa comportamentul algoritmilor în cazuri favorabile și nefavorabile.

2 Istoricul Algoritmilor de Sortare

2.1 Bubble Sort

Origine: Aproximativ 1956.
Istoric: Deși principiul a fost utilizat anterior, termenul de „Bubble Sort" a fost menționat pentru prima dată de Edward H. Friend în 1956. Este unul dintre cei mai simpli algoritmi, dar și cel mai puțin eficient, fiind utilizat astăzi mai mult în scopuri educaționale decât în producție.

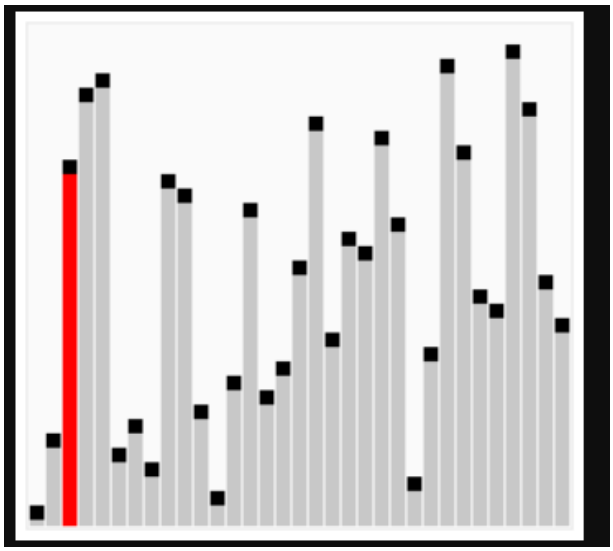

Figura 1: Vizualizare Bubble Sort (Click pe imagine pentru animație)

2.2 Selection Sort

Origine: Anii '50.
Istoric: Este un algoritm intuitiv care a apărut odată cu primele sisteme de calcul. Deși nu are un singur inventator creditat, a fost formalizat în literatura de specialitate la mijlocul secolului XX ca o metodă de reducere a numărului de scrieri în memorie, având o complexitate constantă de $O(n^2)$ schimbări.

2.3 Insertion Sort

Inventator: John Mauchly.
Origine: 1946.

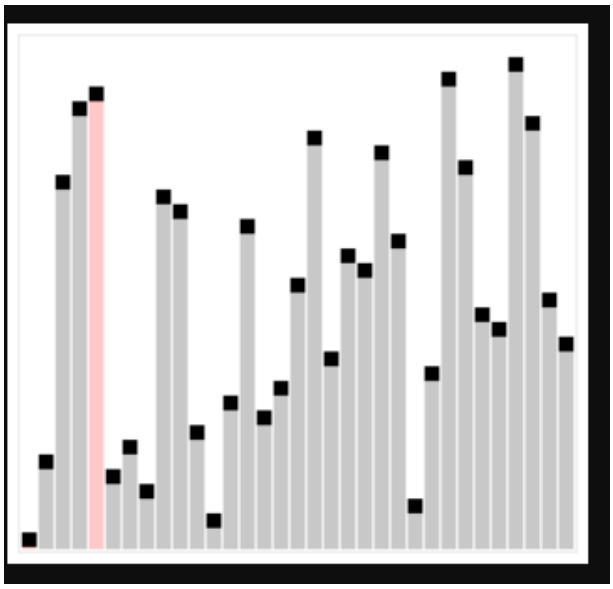

Figura 2: Vizualizare Selection Sort (Click pe imagine pentru animație)

Istoric: A fost unul dintre primii algoritmi de sortare descriși pentru calculatoarele electronice (ENIAC). Mauchly l-a prezentat ca o metodă naturală de sortare, similară modului în care un om sortează o mână de cărți de joc.

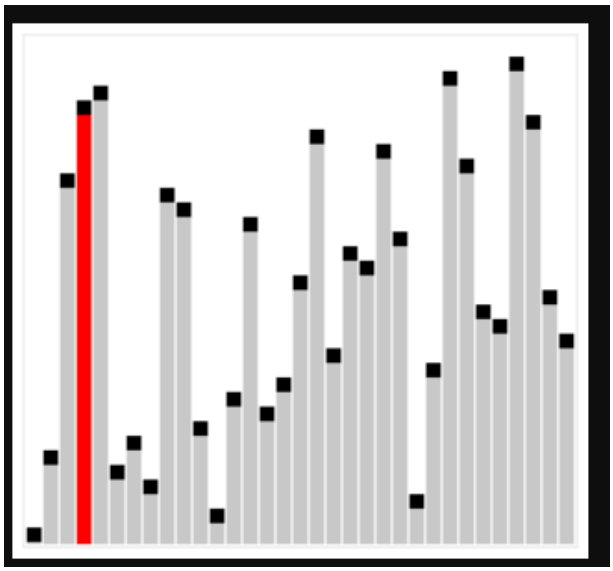

Figura 3: Vizualizare Insertion Sort (Click pe imagine pentru animație)

2.4 Merge Sort

Inventator: John von Neumann.
Origine: 1945.
Istoric: Este primul algoritm de tip *Divide and Conquer*. Von Neumann l-a conceput pentru a demonstra puterea recursivității. Rămâne și astăzi standardul pentru sortarea stabilă și este utilizat ca bază pentru algoritmi moderni precum Timsort.

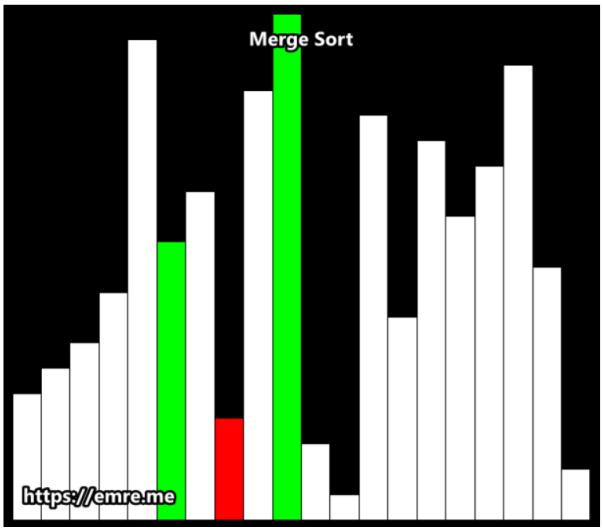

Figura 4: Vizualizare Merge Sort (Click pe imagine pentru animație)

2.5 Quick Sort

Inventator: Sir Tony Hoare.
Origine: 1959.
Istoric: Hoare a dezvoltat Quick Sort în timp ce lucra la un proiect de traducere automată în Uniunea Sovietică. Deși are un caz defavorabil de $O(n^2)$, în practică este adesea cel mai rapid algoritm de sortare pe memoria RAM, fiind baza funcției `std::sort` din C++.

Figura 5: Vizualizare Quick Sort (Click pe imagine pentru animație)

3 Structura Proiectului

Codul este organizat modular pentru a asigura extensibilitatea și claritatea:

- `main.cpp`: Gestionează interfața cu utilizatorul și logica meniului.
- `Algorithm.h`: Conține implementările efective ale celor 5 algoritmi.
- `Generators.h`: Implementează logica pentru generarea seturilor de date: *Random, Sorted, Reverse, Almost Sorted, Flat.*
- `Benchmark.h`: Motorul de testare responsabil pentru cronometrare (utilizând `std::chrono`) și exportul datelor în format CSV.

4 Rezultate Benchmark

Testele au fost efectuate pe un set de **10.000 de elemente**. Valorile din tabelul de mai jos reprezintă timpul de execuție în milisecunde (ms).

Distribuție	Bubble	Selection	Insertion	Merge	Quick
Random	657,44	188,59	213,15	3,57	1,89
Sorted	0,03	188,06	0,08	1,90	0,56
Reverse	749,53	273,96	544,90	1,90	0,33
Almost Sorted	334,76	262,14	36,69	1,90	1,90
Flat	514,31	253,69	181,93	1,90	47,39

Tabela 1: Comparativ performanță (10.000 elemente)

5 Analiza Anomaliilor

5.1 Anomalia Quick Sort pe date „Flat”

În cadrul testelor pe date de tip „Flat” (toate elementele egale), s-a observat o degradare a performanței Quick Sort (47,39 ms față de 1,89 ms pe date random).

Explicație: Această degradare apare deoarece pivotul ales nu reușește să partiționeze lista în sub-liste echilibrate. Dacă toate elementele sunt egale cu pivotul, algoritmul plasează adesea 0 elemente într-o parte și $n - 1$ în cealaltă, transformând complexitatea ideală de $O(n \log n)$ într-una degenerată de $O(n^2)$.

5.2 Anomalia Insertion Sort pe date „Flat”

Deși teoretic Insertion Sort ar trebui să aibă o performanță de $O(n)$ pe date egale, în testele noastre a înregistrat 181,93 ms.

Explicație: Această problemă ține de implementarea operatorului logic în bucla de căutare. Dacă se folosește `while(arr[j] >= cheie)` în loc de `while(arr[j] > cheie)`, algoritmul va efectua schimbări inutile pentru elemente identice, generând operații de scriere redundante în memorie.

6 Concluzii

Rezultatele confirmă superioritatea algoritmilor $O(n \log n)$ (Merge și Quick Sort) pentru seturi mari de date. Totuși, se observă că algoritmi simpli precum Insertion Sort sunt extrem de eficienți pe date deja sortate sau aproape sortate, depășind chiar și Quick Sort în anumite scenarii specifice.

7 Referințe

- Emre.me, *Sorting Algorithms*:
  https://emre.me/algorithms/sorting-algorithms/
- Wikimedia Commons (sursă pentru animații):
  https://commons.wikimedia.org/wiki/Main_Page
- Wikipedia, *Bubble sort* (Istoric și origini):
  https://en.wikipedia.org/wiki/Bubble_sort
- Wikipedia, *Selection sort* (Istoric):
  https://en.wikipedia.org/wiki/Selection_sort
- Wikipedia, *Insertion sort* (Istoric):
  https://en.wikipedia.org/wiki/Insertion_sort
- Wikipedia, *Merge sort* (John von Neumann, 1945):
  https://en.wikipedia.org/wiki/Merge_sort
- Wikipedia, *Quicksort* (Tony Hoare, 1959):
  https://en.wikipedia.org/wiki/Quicksort